# FINAL PROJECT REPORT

**ON**

# CAMPUS RESOLVE

## Campus Complaint Management System

"Report. Track. Resolve."

Submitted in partial fulfilment of the requirements for the academic project

## Submitted by

G. Dheeraj (25MVCSMR0394)

G. Vishnu Vardhan Reddy (25MVCSMR0398)

Chinna Raju (25MVCSMR0424)

Shaik Shareef (25MVCSMR0440)

Department: CSE (Artificial Intelligence and Machine Learning)

College: Malla Reddy Technical Campus

Academic Year: 2026–2027

# DECLARATION

We hereby declare that the project entitled “CampusResolve – Campus Complaint Management System” has been prepared by our team as an academic project. The report describes the proposed system, its design, implementation approach, and verification plan. Any demonstration data and test outcomes must be checked against the running application before final submission.

Team Members: G. Dheeraj | G. Vishnu Vardhan Reddy | Chinna Raju | Shaik Shareef

Date: 10-10-2026 Place: Malla Reddy Technical Campus

# ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide and department for their guidance and support. We also thank our college for providing an opportunity to develop this project. Finally, we thank our team members for their cooperation in planning, implementation, testing, and documentation.

# ABSTRACT

CampusResolve is a web-based Campus Complaint Management System designed to make reporting and resolving campus issues more transparent and accountable. Students can register or log in, submit complaints with a category, campus location, urgency level, description, and optional photo evidence, and track progress using a unique reference ID. Administrators can view campus-wide complaints, prioritize urgent issues, assign them to the appropriate department, and update their status with remarks. The system uses a MERN-stack architecture: React for the user interface, Node.js and Express.js for REST APIs, and MongoDB with Mongoose for persistent storage. JWT authentication, password hashing, and role-based access control are included in the project design. The aim is to replace fragmented paper-based or informal reporting with a consistent digital workflow.

# TABLE OF CONTENTS

1. Introduction
2. Problem Statement, Objectives and Scope
3. Features and Functional Requirements
4. Algorithms and Methodology
5. Technologies and Implementation
6. Results, Testing and Performance
7. Conclusion and Future Scope
8. References

- Appendix A: Setup and Execution
- Appendix B: Items to Complete Before Submission

# 1. INTRODUCTION

## 1.1 Project Overview

CampusResolve provides one digital location where students can report infrastructure and service issues such as damaged classroom equipment, laboratory problems, washroom maintenance, electrical faults, campus Wi-Fi problems, and cleanliness concerns. Each complaint is assigned a unique reference number and maintains a status history so that students can follow its progress.

## 1.2 Background

In many institutions, complaints may be communicated through paper registers, verbal requests, or informal emails. These approaches can make it difficult to identify the responsible department, confirm receipt, track progress, and review recurring issues. A centralized system can improve traceability and provide administrators with a consolidated view of reported problems.

## 1.3 Proposed Solution

The proposed application separates student and administrator capabilities. Students submit and track their own complaints. Administrators review all complaints available to their role, prioritize urgent matters, and record status changes and remarks. The backend validates requests, checks authorization, and stores records in MongoDB through Mongoose.

## 1.4 Purpose

- Digitize campus complaint submission and follow-up.
- Provide unique complaint references and visible status history.
- Improve accountability through administrator remarks and recorded updates.
- Help administrators identify common categories and locations of campus issues.

# 2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

## 2.1 Problem Statement

Traditional campus complaint handling may suffer from misplaced reports, limited status visibility, unclear ownership, and delayed escalation. Students may not know whether a complaint has been received or acted upon, while administrators may lack a single view of unresolved issues. CampusResolve addresses this problem through a centralized, role-based digital grievance workflow.

## 2.2 Objectives

- Create a simple interface for students to submit campus complaints.
- Generate a unique reference ID for each complaint.
- Allow students to view the status and history of their own complaints.
- Provide administrators with a dashboard for reviewing and managing complaints.
- Support categories, locations, urgency levels, optional attachments, and status remarks.
- Protect accounts and endpoints using authentication and role-based authorization.
- Store complaint and user records persistently in MongoDB.

## 2.3 Project Scope

The project covers account registration and login, complaint creation, complaint listing and tracking, administrator review, status updates, dashboard summaries, and secure access to role-specific functions. The current scope is a web application. Email/SMS notifications, service-level timers, mobile-app packaging, and production cloud deployment are possible future extensions rather than assumed completed features.

### 2.4 Target Users

| User role | Main responsibilities |
|---|---|
| **Student** | Register/login, submit a complaint, provide category/location/urgency, attach evidence where supported, and track own complaints. |
| **Administrator** | Review campus complaints, prioritize urgent issues, assign or route issues, update status, and add official remarks. |
| **Institution / maintenance team** | Use complaint details and status information to coordinate follow-up, according to the institution's process. |

# 3. FEATURES AND FUNCTIONAL REQUIREMENTS

### 3.1 Main Features

- User registration and login.
- Role-based student and administrator dashboards.
- Complaint submission with category, location, urgency, description, and optional photo attachment.
- Unique complaint reference number in the CMP-2026-XXXXX style.
- Complaint status tracking and status-history timeline.
- Administrator complaint review, prioritization, assignment/routing, and status remarks.
- Dashboard summaries and category/location trends where implemented.
- API validation, protected routes, and controlled access to complaint records.

### 3.2 Functional Requirements

| ID | Requirement | Expected behaviour |
|---|---|---|
| **FR-01** | Registration | Accept valid user details and reject duplicate email addresses. |
| **FR-02** | Authentication | Verify credentials and issue a JWT on successful login. |
| **FR-03** | Complaint creation | Validate required fields and save a complaint with a unique reference ID and initial status. |
| **FR-04** | Student tracking | Allow a student to access only their own complaints. |
| **FR-05** | Admin review | Allow an authorized administrator to retrieve and review campus complaints. |
| **FR-06** | Status update | Record status changes and remarks in the complaint history. |
| **FR-07** | Attachment handling | Accept optional complaint evidence subject to configured upload validation. |
| **FR-08** | Dashboard | Present relevant complaint summaries to the correct user role. |

### 3.3 CRUD Operations

| Operation | Use in the system |
|---|---|
| **Create** | Create a user account and submit a complaint. |
| **Read** | View profile, own complaints, complaint details, status history, and administrator complaint lists. |
| **Update** | Update permitted profile fields and, for administrators, complaint status/remarks or assignment fields supported by the implementation. |
| **Delete** | Deletion is not described as a primary workflow in the supplied project documentation. Include it only if a delete feature is implemented and tested. |

### 3.4 Authentication, Authorization and Validation

The project uses JWT-based authentication and bcryptjs password hashing. Role-based access control separates student routes from administrator-only routes. Request validation should reject missing or invalid fields, duplicate accounts, invalid credentials, unauthorized role changes, and attempts to access another student's complaints. Helmet, CORS configuration, rate limiting, and mass-assignment protection are listed as security measures in the project documentation.

### 3.5 System Requirements

| Category | Requirement |
|---|---|
| **Software** | Node.js 18.x or newer; npm 9.x or newer. The root dependency concurrently v10 requires a recent Node.js version; use a currently supported Node.js release. |
| **Database** | Local MongoDB Community Server or MongoDB Atlas connection. |
| **Frontend** | React 19, Vite 8, Tailwind CSS, React Router, and related UI packages. |
| **Backend** | Node.js, Express.js, Mongoose, JWT, bcryptjs, and configured middleware. |
| **Tools** | Code editor, web browser, terminal/command prompt, Git (recommended). |
| **Hardware** | A computer capable of running Node.js, a browser, and the MongoDB service or accessing MongoDB Atlas. |

## 4. ALGORITHMS AND METHODOLOGY

### 4.1 System Workflow

Student workflow: Register/Login → Open complaint form → Enter issue details → Validate and submit → Receive reference ID → Track status history → Review resolution remarks.

Administrator workflow: Login → Open dashboard → Review complaint queue → Prioritize and route/assign issue → Update status and remarks → Save status history → Student views the updated timeline.

### 4.2 Algorithms / Core Logic

1. Authentication: receive credentials, validate input, find the user record, compare the submitted password with the stored bcrypt hash, and issue a signed JWT if valid.
2. Authorization: validate the JWT on protected requests and check the user's role before allowing access to administrator-only actions.
3. Complaint creation: validate required complaint fields, generate a unique CMP-style reference, set the initial status to Pending, save the record, and return the reference to the client.
4. Status tracking: when an authorized administrator changes a complaint status, record the new status, remarks, and timestamp in status history; record a resolution timestamp when the complaint is marked Resolved.
5. Dashboard summary: retrieve relevant complaint records and calculate counts or groupings used to display complaint statistics.

The supplied project presentation explicitly states that no optimization algorithm is claimed. The main logic is validation, authorization, persistence, and status-history tracking. Any complexity estimates should be treated as general estimates and confirmed against the actual database queries and implementation.

### 4.3 Development Methodology

An iterative development approach is suitable for this project: identify requirements, design the user roles and data model, build the frontend and REST API, connect the database, implement authentication and authorization, test key workflows, and refine the interface based on test findings. The application is divided into frontend, backend, database, and shared API responsibilities.

### 4.4 Data Flow

| Step | Data flow |
|---|---|
| **1. Input** | Student enters complaint details in the React interface. |
| **2. Request** | Frontend sends an HTTP request to the Express REST API. |
| **3. Validation** | Backend middleware/controllers validate the token, role, and request data. |
| **4. Persistence** | Mongoose reads or writes user and complaint documents in MongoDB. |
| **5. Response** | API returns a result or error response to the frontend. |
| **6. Display** | React displays the complaint reference, status, timeline, or dashboard information. |

## 5. TECHNOLOGIES AND IMPLEMENTATION

### 5.1 Frontend Technologies

- React 19: component-based user interface.
- Vite 8: frontend development server and build tooling.
- Tailwind CSS: styling and responsive layouts.
- React Router v7: navigation and protected role-based routes.
- Axios: API requests, including bearer-token handling through an interceptor.
- Lucide React and Recharts: icons and chart visualizations where used.

### 5.2 Backend Technologies

- Node.js: JavaScript runtime for server-side execution.

- Express.js: REST API routing and middleware.
- Mongoose: schema-based interaction with MongoDB.
- JWT and bcryptjs: token authentication and password hashing.
- Multer: handling uploaded files where configured.
- Helmet, CORS, rate limiting, error-handling middleware, and role checks: supporting security and API reliability.

### 5.3 Database

MongoDB is used for persistent data storage. The supplied project structure identifies User, Complaint, and Category models. A complaint record logically needs fields for its reference ID, creator, category, location, urgency, description, current status, attachment metadata where applicable, timestamps, and status-history entries. Exact field names and schema constraints should be verified in the implementation before final submission.

### 5.4 Tools and APIs

The project exposes an Express REST API and documents an API reference at docs/API_DOCUMENTATION.md. The frontend uses a configurable API base URL, normally http://localhost:5000/api during local development. A health-check endpoint is documented at /api/health. Refer to the project's API documentation for the exact endpoint methods, request bodies, and response formats.

### 5.5 Implementation Summary

The codebase is organized into frontend pages, reusable components, authentication context, API services, backend controllers, middleware, models, routes, scripts, and tests. The root package scripts support installing all dependencies, running frontend and backend concurrently, seeding demonstration data, running tests, and creating a production frontend build.

## 6. RESULTS, TESTING AND PERFORMANCE

### 6.1 Expected / Demonstrable Results

When configured and running correctly, the application should allow a student to register or log in, submit a complaint, receive a CMP-style reference ID, and view its status history. An administrator should be able to review the complaint and update its status with remarks. These outcomes must be demonstrated with the running application and supported by actual screenshots before the report is submitted.

### 6.2 Test Cases

| Test ID | Test scenario | Expected result | Actual result |
|---|---|---|---|
| **TC-01** | Register with valid details | Account is created successfully. | To be filled after testing |
| **TC-02** | Register using an existing email | Request is rejected with a validation error. | To be filled after testing |
| **TC-03** | Login with valid credentials | Login succeeds and an authentication token is issued. | To be filled after testing |
| **TC-04** | Login with invalid credentials | Access is denied. | To be filled after testing |
| **TC-05** | Submit a valid complaint | Complaint is saved with a unique reference ID and Pending status. | To be filled after testing |
| **TC-06** | Student opens another | Access is blocked. | To be filled after testing |

| | | | |
|---|---|---|---|
| | student's complaint | | |
| **TC-07** | Student calls an administrator-only endpoint | Access is blocked. | To be filled after testing |
| **TC-08** | Administrator updates complaint status | Status, remarks, and history are saved. | To be filled after testing |
| **TC-09** | Complaint is marked Resolved | Resolution timestamp is recorded where implemented. | To be filled after testing |
| **TC-10** | Attempt to change role during registration | Unauthorized role escalation is rejected. | To be filled after testing |
| **TC-11** | Database is unavailable at startup | Backend reports a connection failure and does not start normally. | To be filled after testing |
| **TC-12** | Upload valid/invalid attachment | Configured file rules are enforced. | To be filled after testing |

The README describes an automated backend suite covering 17 acceptance criteria. Run `npm test` from the project root and record the real pass/fail output here. Do not mark tests as passed unless they were actually executed.

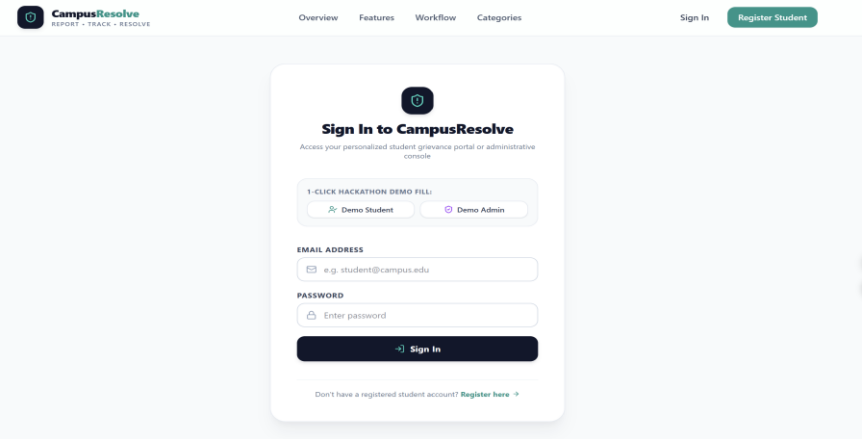
### 6.3 Testing Methods

- Functional testing: verify registration, login, complaint creation, tracking, and administrator updates.
- API testing: verify request methods, response codes, validation, and error responses.
- Authorization testing: verify that students cannot access administrator functions or other students' records.
- Database testing: verify that records persist and status history is retained.
- UI testing: check forms, navigation, responsive layout, loading states, and error messages.
- Regression testing: rerun the automated test suite after code changes.

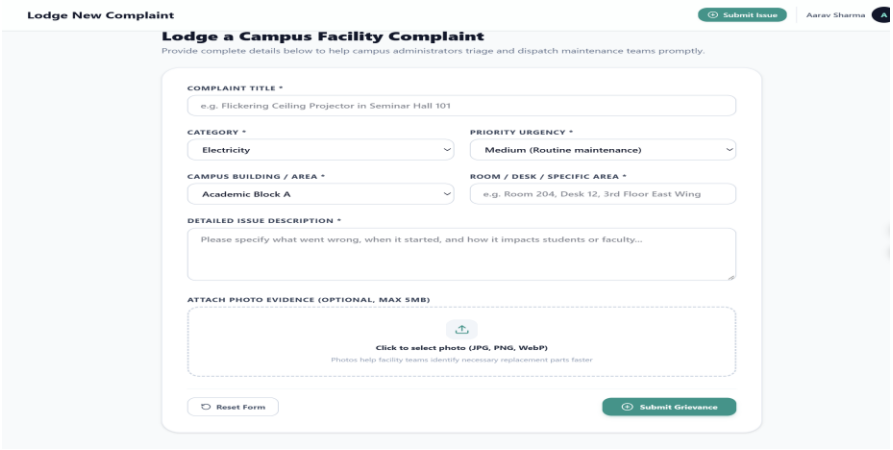
### 6.4 Performance Analysis

The project uses a client-server architecture with database persistence. Actual performance depends on dataset size, indexes, query design, server hardware, network conditions, and concurrent users. No measured response-time, throughput, or load-test results were provided in the supplied materials; therefore, numerical performance claims should not be included until measured. For evaluation, record average and 95th-percentile API latency, error rate, and behaviour under increasing concurrent requests.

### 6.5 Screenshots / Output



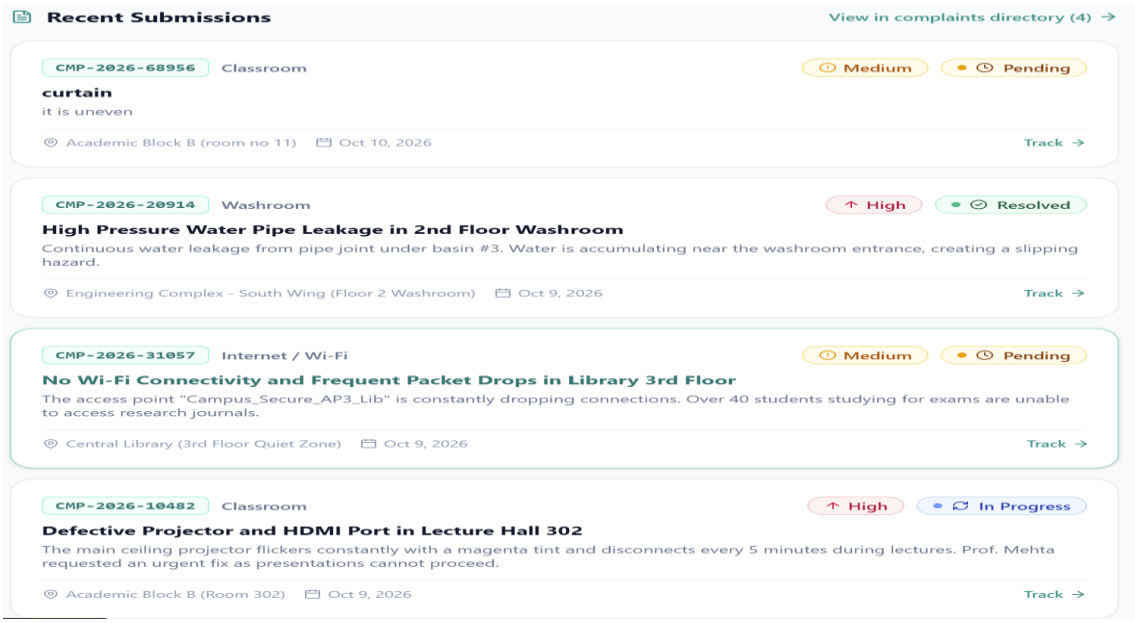

*Student registration/login page*



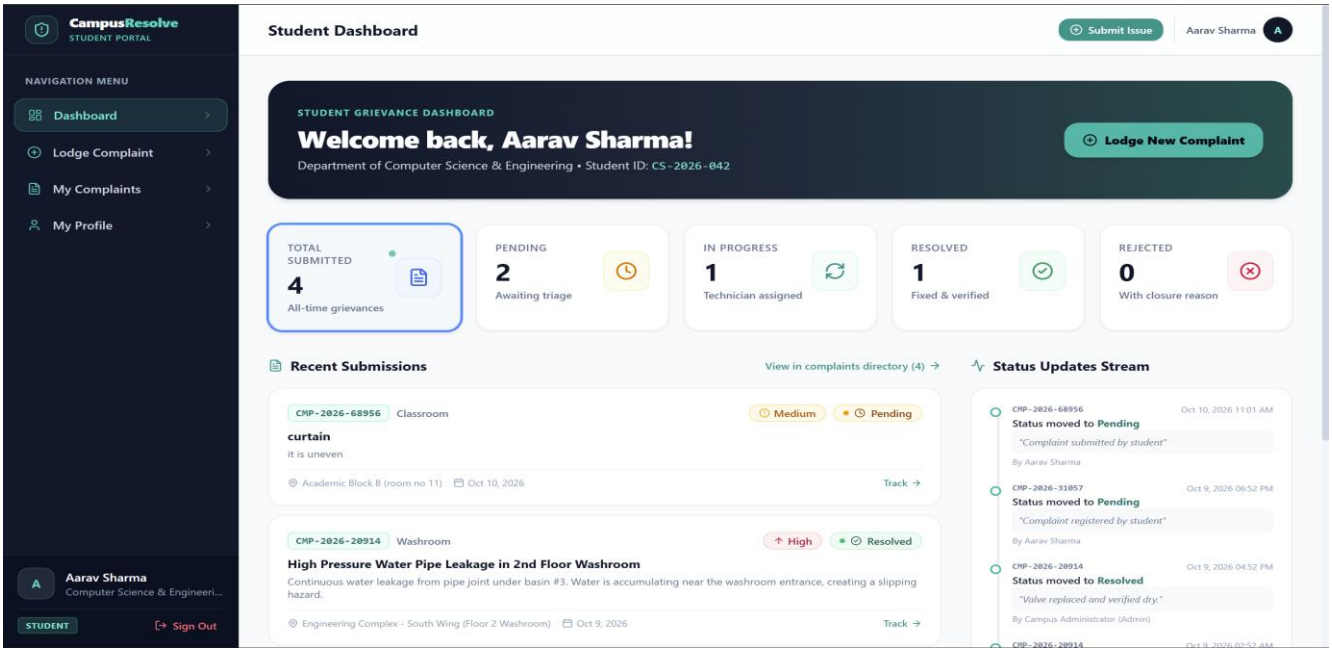

Student complaint submission form




Student complaint list and status timeline




Administrator dashboard and complaint queue




Test execution output

# 7. CONCLUSION AND FUTURE SCOPE

## 7.1 Conclusion

CampusResolve proposes a centralized and role-based approach to campus grievance management. It brings complaint submission, reference-based tracking, administrator review, and status-history recording into a single web application. The MERN stack provides a clear separation between the user interface, API layer, and database. Authentication and authorization help protect student records and administrator operations. The project can reduce dependence on informal reporting channels and improve transparency, provided the application is deployed with appropriate institutional processes and verified through testing.

## 7.2 Future Enhancements

- Email and SMS notifications when complaint status changes.
- Service-level agreement (SLA) timers, reminders, and automatic escalation for overdue complaints.
- Progressive Web App (PWA) support for a mobile-friendly experience.
- More detailed analytics by department, category, building, and resolution time.
- Cloud deployment, scheduled backups, centralized logging, and monitoring.
- Load testing, accessibility audits, and additional security testing.
- Feedback and satisfaction ratings after a complaint is resolved.

# 8. REFERENCES


- CampusResolve project README and setup documentation (provided with the project source).
- React documentation: https://react.dev/
- Vite documentation: https://vite.dev/
- Node.js documentation: https://nodejs.org/docs/
- Express.js documentation: https://expressjs.com/
- MongoDB documentation: https://www.mongodb.com/docs/
- Mongoose documentation: https://mongoosejs.com/docs/
- JSON Web Token introduction: https://jwt.io/introduction
- Tailwind CSS documentation: https://tailwindcss.com/docs


# APPENDIX A: SETUP AND EXECUTION

Prerequisites: install a supported Node.js version, npm, and ensure MongoDB is running or configure MongoDB Atlas.

Open a terminal in the project root folder.

6. Install dependencies using `npm run install:all`.
7. Configure backend environment variables in `backend/.env`, including PORT, MONGODB_URI, JWT_SECRET, JWT_EXPIRES_IN, and CLIENT_URL.
8. Configure the frontend API URL in `frontend/.env`, normally `VITE_API_URL=http://localhost:5000/api`.
9. Initialize demonstration data using `npm run seed` or the documented seed script.
10. Start the application using `npm run dev`.
11. Open the frontend at `http://localhost:5173`; the backend is normally available at `http://localhost:5000`.
12. Run the automated tests with `npm test` and build the frontend with `npm run build`.

Security note: use a private, strong JWT secret for local or deployed environments. Do not publish real secrets or demo passwords in a public repository. Change seeded demonstration credentials before any public deployment.

# APPENDIX B: ITEMS TO COMPLETE BEFORE SUBMISSION

- Fill in the official college name, department, guide name, roll numbers, and confirmed academic year.
- Confirm the team member names and spelling against college records.
- Run the project and replace screenshot placeholders with real application screenshots.
- Run `npm test` and enter the actual test outcomes in the test table.
- *Add measured performance results only if performance testing is performed.*
- *Check that every feature described in the report is present in the submitted code.*
- Review the college's required report format, certificate page, page numbering, and signature requirements.